Bigger Is Greater

Platform: HackerRank
Difficulty: Medium
Topic: Strings, Lexicographical Order, Next Permutation

Problem Link: https://www.hackerrank.com/challenges/bigger-is-greater/problem

Problem Statement

Given a word, create a new word by swapping some or all of its characters. This new word must meet two criteria:

- It must be lexicographically greater than the original word.
- It must be the smallest word that meets the first condition.

Example

For the word `ab`, the next largest word is `ba`.

Complete the function `biggerIsGreater` below to create and return the new string meeting the criteria. If it is not possible, return `no answer`.

Function Description

Complete the `biggerIsGreater` function.

Parameter

- `string w`: a word.

Returns

- `string`: the smallest lexicographically greater string possible, or `no answer` if no such string exists.

Input Format

The first line contains an integer `T`, the number of test cases.

Each of the next `T` lines contains a string `w`.

Constraints

- `1 <= T <= 10^5`
- `1 <= length of w <= 100`
- `w` contains only lowercase English letters in the range `a` to `z`.

Sample Input 0

```
5
ab
bb
hefg
dhck
dkhc
```

Sample Output 0

```
ba
no answer
hegf
dhkc
hcdk
```

Explanation 0

- **Test case 1:** `ba` is the only string that can be made by rearranging `ab`. It is greater than the original string.
- **Test case 2:** It is not possible to rearrange `bb` to obtain a greater string.
- **Test case 3:** `hegf` is the next string greater than `hefg`.
- **Test case 4:** `dhkc` is the next string greater than `dhck`.
- **Test case 5:** `hcdk` is the next string greater than `dkhc`.

Java Method Signature

```java
public static String biggerIsGreater(String w) {
    // Write your code here
}
```

My Solution

```
// Write your solution here
```

Complexity Analysis

- **Time Complexity:** To be determined after implementation.
- **Space Complexity:** To be determined after implementation.